

DEADLOCK GUARDIAN

Deadlock Risk Detection, Detection & Recovery System

İşletim Sistemleri Dersi — Proje Tanım ve 11 Haftalık Yol Haritası

Projenin amacı	Programların kaynakları kullanırken birbirlerini kilitlemesini (deadlock) önceden fark etmek, gerçekleştiğinde tespit etmek ve sistemin toparlanması için çözüm sunmak.
Temel OS konuları	Process, Resource Allocation, Deadlock Detection, Resource Allocation Graph, Banker's Algorithm, Recovery ve Synchronization.
Hedef	Sadece bir algoritma değil, deadlock yönetimini kullanıcıya anlaşılır biçimde gösteren çalışan bir sistem geliştirmek.

Bu doküman ekip arkadaşlarının projeyi aynı şekilde anlayabilmesi için hazırlanmıştır.

1. Proje Fikri — Çok Basit Anlatım

Bir bilgisayarda birçok program aynı anda çalışır. Bu programlar aynı kaynakları (dosya, bellek bölgesi, cihaz, kilit vb.) kullanmak isteyebilir. Eğer bir program bir kaynağı alıp başka bir kaynağı beklerken, diğer program da ikinci kaynağı alıp birincisini beklerse iki taraf da sonsuza kadar bekleyebilir. Buna deadlock denir.

Bizim sistemimiz üç şey yapacak:

Aşama	Ne yapıyor?
1 — Risk Analizi	Deadlock oluşmadan önce tehlikeli bir durum varsa kullanıcıyı uyarır.
2 — Deadlock Tespiti	Kilitlenme oluştuğunda hangi process ve kaynakların probleme neden olduğunu bulur.
3 — Recovery	Deadlock sonrası sistemi tekrar güvenli hale getirmek için çözüm seçenekleri sunar.

2. Basit Örnek

P1 R1 kaynağını almış ve R2'yi bekliyor. P2 R2'yi almış ve R1'i bekliyor. P1, P2'yi; P2 de P1'i beklediği için sistem ilerleyemiyor.

Process	Elindeki	Beklediği
P1	R1	R2
P2	R2	R1

Sistemimiz bu ilişkiyi bir grafik olarak inceleyip $P1 \rightarrow R2 \rightarrow P2 \rightarrow R1 \rightarrow P1$ döngüsünü bulabilir. Döngü, deadlock olduğuna işaret eder.

3. Kullanıcı Programda Ne Görecek?

DEADLOCK GUARDIAN		
Aktif Process 12	Kaynak 8	Deadlock Riski Yüksek
Process Durumu P1 — Çalışıyor P2 — Bekliyor P3 — Çalışıyor	Kaynak Durumu R1 — P1 R2 — P3 R3 — Boş	Uyarı P2 ve P4 karşılıklı kaynak bekliyor.

4. Deadlock Oluşunca Ne Olacak?

Sistem önce probleme karışan process'leri ve kaynakları gösterecek. Ardından farklı kurtarma seçenekleri sunacak. Örneğin bir process'i sonlandırmak onun tuttuğu kaynakları serbest bırakabilir.

İlk sürüm için önemli karar: Gerçek bilgisayardaki tüm process'leri yönetmek yerine kontrollü bir simülasyon ortamında process/resource senaryoları oluşturacağız. Gerçek sistem entegrasyonu daha sonra ek özellik olabilir.

5. Projenin Genel Yapısı

Process'ler	Kaynak ister / kaynak bırakır
↓	↓
Deadlock Guardian	Tüm ilişkileri takip eder
↓	↓
Risk Analizi	Deadlock Detection
↓	↓
Uyarı	Recovery

6. Kullanacağımız İşletim Sistemi Konuları

Konu	Projede ne anlama geliyor?
Process	Bilgisayarda çalışan programları temsil eder.
Resource	Process'lerin kullanmak istediği kaynakları temsil eder.
Resource Allocation	Hangi process'in hangi kaynağı aldığını takip ederiz.
Deadlock	Process'lerin birbirini bekleyip ilerleyemediği durum.
Resource Allocation Graph	Process-kaynak ilişkilerini görsel olarak gösterir.
Cycle Detection	Grafikte döngü olup olmadığını buluruz.
Banker's Algorithm	Kaynak verildiğinde sistemin güvenli durumda kalıp kalmayacağını kontrol etmek için kullanılabilir.
Recovery	Deadlock sonrası sistemi güvenli duruma döndürme yaklaşımı.

7. 11 Haftalık Geliştirme Planı

Hafta	Aşama	Yapılacak iş
1	Konuyu Anlama	Deadlock, process, resource ve waiting kavramlarını öğrenmek; proje kapsamını kesinleştirmek.
2	Basit Simülasyon	Process ve resource oluşturmak; alma, bırakma ve bekleme işlemlerini çalıştırmak.
3	Kaynak İlişkileri	Hangi process hangi kaynağı kullanıyor/bekliyor bilgisini tutmak.
4	Detection	Resource Allocation Graph ve cycle detection ile deadlock bulmak.
5	Görselleştirme	Process-resource ilişkilerini anlaşılır grafikte göstermek.
6	Risk Analizi	Deadlock oluşmadan önce riskli durumları belirlemek.

Hafta	Aşama	Yapılacak iş
7	Risk Sistemi	Low/Medium/High/Critical seviyelerini ve risk nedenlerini göstermek.
8	Recovery	Deadlock sonrası çözüm seçeneklerini hesaplamak.
9	Arayüz	Dashboard, process/resource listeleri, risk ve deadlock ekranlarını yapmak.
10	Test	Normal, tek deadlock, çoklu deadlock ve recovery senaryolarını test etmek.
11	Final	Demo, rapor, sunum ve son hata düzeltmeleri.

8. Ekip İçinde Görev Dağılımı

Aşağıdaki dağılım 3 kişilik ekip için örnektir; ekip sayısı değişirse yeniden düzenlenebilir.

Üye	Ana sorumluluk	Örnek görevler
Üye 1	Deadlock algoritmaları	Detection, cycle detection, risk analizi ve recovery mantığı.
Üye 2	Simülasyon ve veri modeli	Process/resource yapıları, simülasyon motoru ve test senaryoları.
Üye 3	Arayüz ve görselleştirme	Dashboard, grafikler, kullanıcı akışı ve sonuç ekranları.

9. Önerilen Teknoloji

Katman	Öneri	Neden?
Simülasyon	Python	Hızlı geliştirme ve kolay test.
Algoritmalar	Python	Graph ve veri yapıları rahat uygulanır.
Arayüz	PyQt veya basit web arayüzü	Dashboard ve grafik göstermek için.
Veri	JSON / SQLite	Senaryoları ve sonuçları saklamak için.
Ekip çalışması	Git + GitHub	Kod paylaşımı, branch ve sürüm takibi için.

10. Final Demo Senaryosu

1. Sistemi açıyoruz.
2. Üç process ve üç kaynak oluşturuyoruz.
3. Kaynakları process'lere dağıtıyoruz.
4. İki process'in birbirini beklediği senaryoyu oluşturuyoruz.
5. Sistem risk artışını gösteriyor.
6. Deadlock oluşuyor.
7. Sistem hangi process ve kaynakların probleme neden olduğunu gösteriyor.
8. Recovery seçenekleri çıkıyor.
9. Bir process'i sonlandırıyoruz ve kaynakları serbest bırakıyoruz.
10. Sistem tekrar SAFE durumuna geçiyor.

11. Projenin Başarı Kriterleri

- En az bir deadlock senaryosunu doğru tespit etmek.
- Deadlock'a dahil process ve kaynakları göstermek.
- Belirlenen koşullarda deadlock risk uyarısı vermek.
- En az bir recovery yöntemini uygulamak.
- Recovery sonrasında sistemin güvenli duruma döndüğünü göstermek.
- Normal ve deadlock durumlarını karşılaştırmak.
- Kodun ekip üyelerinin ayrı çalışabileceği şekilde modüler olması.
- Finalde baştan sona çalışan bir demo sunmak.

Tek cümlelik proje tanımı: Deadlock Guardian; process'lerin kaynak kullanımını izleyen, deadlock riskini değerlendiren, oluşan deadlock'u tespit eden ve sistemi yeniden güvenli hale getirmek için recovery

seenekleri sunan bir iřletim sistemi projesidir.